

Legal Compliance Assistant

An AI-assisted platform for extracting regulatory requirements, processing internal documents, mapping requirements to organizational evidence, and producing auditor-friendly compliance reports.

The project implements a **naive-to-advanced Retrieval-Augmented Generation (RAG) pipeline** designed for compliance gap analysis. It combines document parsing, hierarchical chunking, multilingual embeddings, lexical retrieval, section-aware scoring, LLM analysis, and experimental knowledge-graph retrieval.

[!IMPORTANT]

This repository is a research and development prototype. Human review remains necessary for legal, regulatory, and audit decisions.

Table of contents

- [Overview](#)
- [Main features](#)
- [How it works](#)
- [Architecture](#)
- [Retrieval strategy](#)
- [Technology stack](#)
- [Repository structure](#)
- [Getting started](#)
- [Configuration](#)
- [Running the application](#)
- [Application workflow](#)
- [API overview](#)
- [GraphRAG experiments](#)
- [Generated outputs](#)
- [Security notes](#)
- [Known limitations](#)

- [Roadmap](#)
- [Contributing](#)
- [License](#)

Overview

Compliance assessments often require comparing a large regulatory document with many internal policies, procedures, manuals, and controls. Manual review is slow, repetitive, and difficult to reproduce.

Legal Compliance Assistant supports this process through four connected workflows:

1. **Requirement extraction** — identifies structured obligations, audit questions, references, sections, and source pages from regulatory documents.
2. **Internal-document processing** — converts internal documents into hierarchical, overlapping, context-preserving chunks.
3. **Compliance mapping** — retrieves the strongest evidence for each requirement and asks an LLM to assess coverage, gaps, and recommended actions.
4. **Annex integration** — retrieves relevant annex passages and merges them into the appropriate sections of a primary document.

The main output is not a generic chatbot response. It is a structured **requirement-to-evidence assessment** that can be reviewed and exported for audit work.

Main features

- Regulatory requirement extraction from PDF, Markdown, and pre-extracted TXT input
- Dynamic macro-section detection using an LLM
- Table-aware document processing through Docling
- Hierarchical chunking with overlap and section context
- Page and source provenance preservation
- Multilingual sentence embeddings
- Hybrid semantic, TF-IDF, and keyword retrieval
- Rule-based requirement classification
- Section-aware relevance boosting

- LLM-assisted compliance coverage analysis
- Semantic redundancy detection and requirement consolidation
- JSON, text, Markdown, PDF, and formatted Excel outputs
- Background processing with progress polling
- Mapping history and result inspection through a React interface
- Experimental HippoRAG-style entity graphs and Personalized PageRank
- Static and interactive graph visualizations
- Annex retrieval and LLM-assisted document merging

How it works



Requirement extraction

The regulatory-document pipeline:

1. Converts a document into page-level text and tables.
2. Detects macro-sections in page batches.
3. Produces structured section chunks.
4. Extracts requirements and audit questions with the configured LLM.
5. Preserves source document, chunk, page, section, and references.
6. Groups semantically similar requirements and optionally merges redundancies.

Internal-document processing

The internal-document pipeline:

1. Converts uploaded files with Docling.
2. detects explicit or conventional section headings.
3. Reconstructs document hierarchy.
4. Removes low-value front matter and index-like content.

5. Creates overlapping chunks with parent-section context.
6. Saves chunks in JSON and text formats for later retrieval.

Compliance mapping

For every requirement, the active mapper:

1. Embeds the original and paraphrased requirement.
2. Computes dense semantic similarity against internal-document chunks.
3. Computes TF-IDF similarity.
4. Computes keyword overlap.
5. Applies a section relevance boost.
6. Keeps the strongest candidates above the configured threshold.
7. Sends retrieved evidence to an LLM for structured compliance analysis.
8. Exports the results as JSON and Excel.

Architecture

Legal-Compliance-Assistant/

```
├── backend/
│   ├── config/          # Environment-backed settings and prompts
│   ├── routes/          # Flask API blueprints
│   ├── scripts/         # Chunking, annex merging, and utility pipelines
│   ├── services/        # Extraction, retrieval, mapping, and graph logic
│   ├── llm_provider.py   # Ollama, Gemini, and OpenAI abstraction
│   ├── main.py          # Flask application entry point
│   └── requirements.txt  # Python dependencies
├── frontend/
│   ├── public/          # Static assets
│   ├── src/
│   │   ├── components/  # Upload, status, results, and history views
│   │   ├── App.js       # Main workflow controller
│   │   └── index.js      # React entry point
│   └── package.json      # Frontend dependencies and commands
├── internal/            # Example processed internal-document chunks
├── output/              # Generated requirements, mappings, and reports
└── output_1712/         # Historical experimental outputs
```

Backend responsibilities

Module	Responsibility
backend/main.py	Starts Flask, registers routes, and coordinates application workflows
backend/config/settings.py	Loads models, LLM parameters, API keys, and storage paths
backend/config/prompts_config.py	Stores requirement-extraction, deduplication, and mapping prompts
backend/llm_provider.py	Provides a common interface for Ollama, Gemini, and OpenAI
backend/scripts/chunking_dynamic.py	Detects macro-sections and builds regulatory-document chunks
backend/scripts/chunking_improved.py	Builds hierarchical, overlapping internal-document chunks
backend/services/requirements_service.py	Extracts and serializes regulatory requirements
backend/services/redundancy_service.py	Clusters and merges semantically redundant requirements
backend/services/mapping_service.py	Runs the active hybrid retrieval and compliance-mapping pipeline
backend/scripts/annex_merger.py	Retrieves and merges annex content into a primary document

Frontend responsibilities

The React frontend provides:

- Document upload and selection
- Requirement and chunk management
- Processing progress and status polling
- Mapping execution
- Coverage filtering and result expansion
- Mapping history
- Requirements and Excel inspection
- Annex-merging controls

Retrieval strategy

The active mapping service combines three retrieval signals:

retrieval score = $0.50 \times \text{semantic similarity}$
+ $0.25 \times \text{TF-IDF similarity}$

+ 0.15 × keyword overlap

The score is multiplied by a section relevance boost when the chunk section matches the inferred requirement type. The result is normalized, filtered, and reranked before the strongest evidence is passed to the LLM.

Requirement types currently include:

- Documentation
- Training
- Monitoring
- Communication
- Prohibition
- General

The active baseline implementation is located in:

backend/services/mapping_service.py

Technology stack

Backend

- Python
- Flask and Flask-CORS
- Docling and Docling Core
- spaCy
- Sentence Transformers
- scikit-learn
- NumPy and pandas
- OpenPyXL
- Ollama, Gemini, or OpenAI
- NetworkX and Personalized PageRank in experimental graph modules

Frontend

- React 18
- React Scripts
- Lucide React
- SheetJS (xlsx)
- Tailwind/PostCSS tooling

Default models

Purpose	Default model
Requirement extraction	deepseek-r1:32b
Compliance mapping	deepseek-r1:14b
Multilingual embeddings	paraphrase-multilingual-MiniLM-L12-v2
Local provider	Ollama

All models can be changed through environment variables.

Repository structure

The source tree includes a stable baseline mapper and several research variants.

```
backend/services/  
├── mapping_service.py          # Active baseline  
├── mapping_service_graph1.py   # Graph-augmented experiment  
├── mapping_service_hippo.py    # HippoRAG experiment  
├── mapping_service_hippofull.py # LLM-built full graph experiment  
├── mapping_service_hippolite.py # Lightweight graph experiment  
├── mapping_service_hippolite1.py # Persisted graph/PPR experiment  
├── hippo*.py                  # Graph construction and retrieval variants  
├── requirements_service.py  
├── redundancy_service.py  
├── document_service.py  
└── smart_content_filter.py
```

[!NOTE]

backend/main.py currently imports backend/services/mapping_service.py. The GraphRAG variants are experimental and are not enabled by the main application by default.

Getting started

Prerequisites

Install the following before running the project:

- Python 3.10 or 3.11
- Node.js 18 or newer
- npm
- Git
- Ollama for the default local setup, or credentials for Gemini/OpenAI
- Sufficient disk space for embedding and LLM models

Large DeepSeek models require significant RAM or GPU memory. For development on smaller machines, set the extraction and mapping model variables to lighter Ollama models.

Clone the repository

```
git clone https://github.com/Ashesh27/Legal-Compliance-Assistant.git
cd Legal-Compliance-Assistant
```

Backend environment

```
cd backend
python -m venv .venv
```

Activate the environment:

```
# Linux/macOS
source .venv/bin/activate
```

```
# Windows PowerShell
.venv\Scripts\Activate.ps1
```

Dependency-file correction

The current backend/requirements.txt contains two dependencies joined on one line:

```
xhtml2pdf==0.2.14tqdm==4.66.1
```

Separate them before installation:

```
xhtml2pdf==0.2.14  
tqdm==4.66.1
```

Then install the backend dependencies:

```
pip install --upgrade pip  
pip install -r requirements.txt
```

For the optional graph and visualization experiments, also install:

```
pip install networkx matplotlib pyvis
```

Install the Italian spaCy model used by hierarchical chunking and lightweight graph extraction:

```
python -m spacy download it_core_news_sm
```

Some PDF-rendering backends may require additional operating-system packages. Consult the WeasyPrint installation guide for your platform if PDF export fails.

Local Ollama setup

Start Ollama and download the default models:

```
ollama serve
```

In another terminal:

```
ollama pull deepseek-r1:14b  
ollama pull deepseek-r1:32b
```

To use smaller models, download them and update the corresponding environment variables.

Frontend environment

```
cd ../frontend
npm install
```

Create or update frontend/.env:

```
REACT_APP_API_URL=http://localhost:5020
```

Do not commit machine-specific IP addresses or secrets.

Configuration

Create backend/.env with the settings required by your chosen provider:

```
# Provider: ollama, gemini, or openai
LLM_PROVIDER=ollama

# Models
LLM_MAPPING_MODEL=deepseek-r1:14b
LLM_EXTRACTION_MODEL=deepseek-r1:32b
EMBEDDING_MODEL=paraphrase-multilingual-MiniLM-L12-v2

# Generation parameters
LLM_MAPPING_TEMPERATURE=0.1
LLM_EXTRACTION_TEMPERATURE=0.0001
LLM_NUM_PREDICT=4096
LLM_NUM_CTX=8192
LLM_TOP_P=0.95

# Required only for hosted providers
GEMINI_API_KEY=
OPENAI_API_KEY=

# Runtime folders, relative to backend/
UPLOAD_FOLDER=./uploads
OUTPUT_FOLDER=./output
CHUNKS_FOLDER=./output/chunks
REQUIREMENTS_FOLDER=./output/requirements
INTERNAL_DOCS_FOLDER=./output/internal_docs
MAPPINGS_FOLDER=./output/mappings

# Used only by selected experimental graph modules
```

```
USE_HIPPORAG=false
```

Never commit `.env` files containing secrets.

Running the application

Start the backend

From the `backend` directory with the virtual environment activated:

```
python main.py
```

The Flask API starts at:

```
http://localhost:5020
```

Start the frontend

In a second terminal:

```
cd frontend  
npm start
```

The React application normally opens at:

```
http://localhost:3000
```

Production note

The supplied shell scripts contain machine-specific absolute paths and local TLS filenames. Update them before use, or replace them with environment-driven deployment scripts. Do not use the Flask development server as a public production server.

Application workflow

Extract regulatory requirements

1. Open the requirements module.

2. Enter a name for the extraction.
3. Upload a regulatory PDF, Markdown document, or pre-extracted TXT file.
4. Wait for macro-section detection, extraction, and redundancy elimination.
5. Review or download the generated requirements.

Process internal documents

1. Open the internal-documents module.
2. Upload one or more organizational documents.
3. Wait for conversion and hierarchical chunking.
4. Select the processed chunk collections to use as evidence.

Run compliance mapping

1. Select one extracted requirements set.
2. Select one or more internal-document chunk sets.
3. Provide a mapping name.
4. Start the mapping job.
5. Review coverage, evidence, gaps, actions, and confidence information.
6. Download the formatted Excel report.

Merge annexes

1. Select a primary document.
2. Add one or more annex documents.
3. Start the merge process.
4. The system retrieves relevant annex chunks for each primary section.
5. Download the merged Markdown or PDF output when processing completes.

API overview

The frontend communicates with Flask through /api endpoints.

Method	Endpoint	Purpose
--------	----------	---------

POST	/api/process-requirements	Upload and process a regulatory document
GET	/api/status/requirements	Read extraction status
GET	/api/get-available-requirements	List extracted requirement sets
POST	/api/process-internal-docs	Upload and process internal documents
GET	/api/status/internal_docs	Read document-processing status
GET	/api/get-available-chunks	List processed chunk sets
POST	/api/process-mapping	Start requirement-to-evidence mapping
GET	/api/status/mapping	Read mapping status
GET	/api/get-available-mappings	List saved mappings
GET	/api/get-mapping-results	Load mapping results
GET	/api/download-mapping-excel	Download a mapping workbook
POST	/api/process-annex-merge	Start annex integration
GET	/api/check-annex-status/<job_id>	Poll annex job status
GET	/api/download-annex-result/<job_id>	Download merged output

This is not a complete formal API specification. Review `backend/main.py` and `backend/routes/` for the current request fields and response schemas.

GraphRAG experiments

The repository contains several graph-retrieval prototypes inspired by associative-memory and HippoRAG-style retrieval.

The general experimental flow is:

Internal chunks

- > entity extraction
- > chunk/entity graph
- > query entity matching
- > Personalized PageRank
- > graph-derived chunk scores
- > fusion with baseline retrieval

Two broad variants are present:

- **Lightweight graph retrieval** — uses spaCy entities and noun phrases, local embeddings, NetworkX, and Personalized PageRank.
- **Full LLM graph retrieval** — asks an LLM to extract entities and relations, resolves similar entities, persists the graph and entity embeddings, and retrieves through graph propagation.

Graph modules also generate PNG, GraphML, pickle, and interactive HTML visualizations.

These implementations are research variants. Before enabling one in the main application:

1. Select one canonical graph implementation.
2. Define a stable chunk and entity schema.
3. Separate graph retrieval from the mapping/export pipeline.
4. Add a configurable fusion policy.
5. Evaluate baseline versus graph retrieval on labelled queries.
6. Measure Recall@K, MRR or nDCG, evidence precision, latency, and LLM faithfulness.

Generated outputs

Depending on the workflow, the application creates:

- Dynamic chunks in TXT and JSON
- Concept-enriched chunk JSON
- Chunk metadata
- Raw and deduplicated requirements
- Requirement metadata
- Mapping results in JSON
- Auditor-oriented Excel workbooks
- Text mapping reports
- Merged annex documents in Markdown or PDF
- Graph files and visualizations
- Experimental ablation workbooks

Runtime data should normally remain outside version control.

Security notes

[!CAUTION]

A TLS private key is currently present in the repository under `frontend/`. Treat it as compromised: revoke or rotate it, remove it from the complete Git history, and replace it with an untracked local certificate or a managed secret.

Before deploying this application:

- Remove committed `.env`, certificate, private-key, log, build, cache, output, graph, embedding, and workbook files.
- Rotate any credentials or certificates that have ever been committed.
- Restrict CORS to the expected frontend origin.
- Add authentication and authorization.
- Associate jobs and stored documents with an authenticated user or tenant.
- Validate upload extensions, MIME types, and file sizes.
- Resolve and validate every requested file path against an approved storage root.
- Do not accept arbitrary paths for download or deletion.
- Add malware scanning where untrusted files may be uploaded.
- Run the service behind a production WSGI server and reverse proxy.
- Keep legal and compliance conclusions subject to qualified human review.

Known limitations

- The active application uses the baseline mapper; GraphRAG files are experimental alternatives.
- Several mapping and graph files duplicate large sections of code.
- Some endpoint responsibilities exist in both `main.py` and route blueprints.
- Job state is stored in process memory and is lost on restart.
- Background work uses local threads rather than a durable task queue.
- Embeddings are recomputed instead of being stored in a reusable vector index.
- There is no complete automated backend or frontend test suite.
- There is no labelled retrieval benchmark committed with the project.

- Startup scripts and frontend configuration contain machine-specific paths or addresses.
- The dependency file requires correction and does not list every optional graph dependency.
- The current API does not implement authentication or tenant isolation.

Roadmap

Recommended priorities:

- ☐ Rotate and purge the committed TLS private key
- ☐ Remove generated artifacts and secrets from version control
- ☐ Add a complete `.env.example`
- ☐ Repair and lock backend dependencies
- ☐ Consolidate API routes into one blueprint-based implementation
- ☐ Add filesystem containment and upload validation
- ☐ Add authentication and user/job ownership
- ☐ Introduce a durable background job queue
- ☐ Extract dense, lexical, keyword, and graph retrieval behind one interface
- ☐ Persist embeddings and graph indexes by corpus version
- ☐ Select one canonical GraphRAG implementation
- ☐ Add unit, integration, API, and frontend tests
- ☐ Add a labelled retrieval evaluation set
- ☐ Add CI for linting, tests, secret scanning, and dependency checks
- ☐ Add containerized development and deployment

Contributing

Contributions are welcome after the repository has a defined contribution policy.

Suggested development process:

1. Create a feature branch.
2. Keep changes focused and avoid adding generated outputs.

3. Add or update tests.
4. Document configuration and schema changes.
5. Run backend and frontend checks locally.
6. Open a pull request describing the problem, approach, and validation results.

For retrieval changes, include before-and-after measurements rather than relying only on example queries.

License

No license file is currently included. Until a license is added, the repository remains under default copyright restrictions. Add an appropriate license before expecting external use, modification, or redistribution.

Legal Compliance Assistant is intended to support compliance professionals by improving evidence discovery and report preparation. It does not replace legal advice, regulatory interpretation, or professional audit judgment.